Lab-8

Dining Philosopher problem

```
#include<stdio.h>
#include<stdlib.h>
#include<pthread.h>
#include<semaphore.h>
#include<unistd.h>

#define N 5
sem_t chopstick[N];
pthread_mutex_t mutex;

void *philosopher(void *num){
  int id = *(int*)num;

  while(1){
    pthread_mutex_lock(&mutex);
    printf("philosopher %d status: Thinking\n", id);
    pthread_mutex_unlock(&mutex);

    sleep(1);

    pthread_mutex_lock(&mutex);
    printf("philosopher %d status: waiting\n", id);
    pthread_mutex_unlock(&mutex);

    sem_wait(&chopstick[id]);
    sem_wait(&chopstick[(id+1)%N]);

    pthread_mutex_lock(&mutex);
    printf("philosopher %d status: Eating\n", id);
    pthread_mutex_unlock(&mutex);

    sleep(3);

    sem_post(&chopstick[id]);
    sem_post(&chopstick[(id+1)%N]);
  }
}

int main(){
  pthread_t tid[N];
  int i, ids[N];

  pthread_mutex_init(&mutex, NULL);
```

```
for(i=0;i<N;i++)
  sem_init(&chopstick[i],0,1);

for(i=0;i<N;i++){
  id[i]=i;
  pthread_create(&tid[i],NULL,philosopher,&id[i]);
}
for(i=0;i<N;i++)
  pthread_join(tid[i],NULL);

return 0;
}
```

O/P:

```
Philosopher 0 status : thinking
            2 status : thinking
            1 status : thinking
            3 status : thinking
            4 status : thinking
            2 status : waiting
            4 status : waiting
            4 status : Eating
            1 status : waiting
            0 status : waiting
            2 status : Eating
            3 status : waiting
            4 status : thinking
            1 status : Eating
            2 status : thinking
```

2. Deadlock detection algorithm

```
#include <stdio.h>
#include <stdbool.h>

#define P 10
#define R 10

int allocation[P][R], request[P][R], available[R];
int n, m;

void detectDeadlock(){
  bool finish[P];
  int work[R];
  for(int i=0; i<n; i++) finish[i] = false;
  for(int i=0; i<m; i++) work[i] = available[i];

  bool deadlock = false;

  for(int count=0; count<n; count++){
    bool found = false;
    for(int p=0; p<n; p++){
      if(!finish[p]){
        bool canProceed = true;
        for(int r=0; r<m; r++){
          if(request[p][r] > work[r]){
            canProceed = false;
            break;
          }
        }
        if(canProceed){
          for(int r=0; r<m; r++)
            work[r] += allocation[p][r];
          finish[p] = true;
          found = true;
        }
      }
    }
    if(!found) break;
  }

  for(int p=0; p<n; p++){
    if(!finish[p]){
      deadlock = true;
      printf("Process %d is in deadlock.\n", p);
    }
  }
}
```

```
    if(!deadlock)
      printf("No deadlock detected.\n");
}
int main(){
printf("Enter number of processes:");
scanf("%d", &n);
printf("Enter number of resources:");
scanf("%d", &m);
printf("Enter Allocation Matrix(%d x %d):\n", n, m);
for(int i=0;i<n; i++)
  for(int j=0; j<m; j++)
    scanf("%d", &allocation[i][j]);
  printf("Enter Request Matrix(%d x %d):\n", n, m);
  for(int i=0;i<n;i++)
    for(int j=0;j<m;j++)
      scanf("%d", &request[i][j]);
  Printf("Enter Available Resources(%d):\n", m);
  for(int i=0;i<m;j++)
    scanf("%d", &available[i]);

  detectDeadlock();

  return 0;

}
```

O/P.

Enter number of processes: 3
Enter number of resources: 3
Enter Allocation Matrix (3x3):

0 1 0
2 0 0
3 0 3

Enter Request Matrix (3x3):

0 0 1
0 2 0
0 0 2

Enter Available Resources(3):

0 0 0

Process 0 is in deadlock
Process 1 is in deadlock
Process 2 is in deadlock

LAB - 8

Bankers Algorithm for deadlock avoidance

```
# include < stdio.h >
void inputMatrices (int n, int m, int alloc[n][m], int max[n][m], int avail[m]){
    Printf("Enter allocation Matrix (%d x %d) : \n", n, m);
    for(int i=0; i<n; i++)
        for(int j=0; j<m; j++)
            Scanf("%d", &alloc[i][j]);
    Printf("Enter Max Matrix (%d x %d) : \n", n, m);
    for(int i=0; i<n; i++)
        for(int j=0; j<m; j++)
            Scanf("%d", &Max[i][j]);
    Printf("Enter Available Resources (%d values) : \n", m);
    for(int i=0; i<m; i++)
        Scanf("%d", &avail[i]);
}
void calculateNeed (int n, int m, int alloc[n][m], int max[n][m],
                    int need[n][m]){
    for(int i=0; i<n; i++)
        for(int j=0; j<m; j++)
            need[i][j] = max[i][j] - alloc[i][j];
}
void bankersAlgo (int n, int m, int alloc[n][m], int need[n][m],
                  int avail[m]){
    int f[n], ans[n], ind=0;
    for(int j=0; i<n; i++) f[i]=0;
    for(int k=0; k<n; k++){
        for(int i=0; i<n; i++){
            if(f[i]==0){
                int flag = 0;
                for(int j=0; j<m; j++){
                    if(need[i][j] > avail[j]){
                        flag = 1;
                        break;
                    }
```

```
            if(flag==0){
                ans[ind++]=i;
                for(int y=0; y<m; y++) avail[y]+=alloc[i][y];
                f[i]=1;
            }
        }
    }
    printf("\nSafe sequence: ");
    for(int i=0; i<n-1; i++) printf("P%d -> ", ans[i]);
    printf("P%d\n", ans[n-1]);
}
int main(){
    int n, m;
    printf("Enter number of process and resources: \n");
    scanf("%d %d", &n, &m);
    int alloc[n][m], max[n][m], avail[m], need[n][m];
    inputMatrices(n, m, alloc, max, avail);
    calculateNeed(n, m, alloc, max, need);
    bankersAlgorithm(n, m, alloc, need, avail);
    return 0;
}
```

O/P:

```
Enter number of Processes & resources:
5 3
Enter Allocation matrix (5 x 3):
0 1 0
2 0 0
3 0 2
2 1 1
0 0 2
Enter Max Matrix (5 x 3):
7 5 3
3 2 2
9 0 2
2 2 2
4 3 3
Enter available Resources
3 3 2
```

11/5/26

Safe sequence: $P_1 \rightarrow P_3 \rightarrow P_4 \rightarrow P_0 \rightarrow P_2$

08/5/26

Memory allocation

```
#include <stdio.h>
#include <string.h>
#define MAX_BLOCKS 20
#define MAX_PROCESSES 20

void printAllocation(int processSize[], int n_processes, int allocation[]){
    printf("\n Process No.\t Process Size\t Block No.\n");
    for(int i=0; i<n_processes; i++){
        printf("%d\t\t%d\t\t", i+1, processSize[i]);
        if (allocation[i] != -1){
            printf("%d\n", allocation[i]+1);
        } else {
            printf("Not allocated\n");
        }
    }
}

void firstFit(int blockSize[], int m_blocks, int processSize[], int
    n_processes){
    int allocation[MAX_PROCESSES];
    int blocks[MAX_BLOCKS];

    memcpy(blocks, blockSize, sizeof(blockSize[0]) * m_blocks);
    memset(allocation, -1, sizeof(allocation));
    for(int i=0; i<n_processes; i++){
        for(int j=0; j<m_blocks; j++){
            if(blocks[j] >= processSize[i]){
                allocation[i] = j;
                blocks[j] -= processSize[i];
                break;
            }
        }
    }
    printf("\n---First fit Allocation---");
    printAllocation(processSize, n_processes, allocation);
```

```
void bestFit(int blockSize[], int m_blocks, int processSize[],
    int n_processes){
    int allocation[MAX_PROCESSES];
    int blocks[MAX_BLOCKS];
    memcpy(blocks, blockSize, sizeof(blocks[0]) * m_blocks);
    memset(allocation, -1, sizeof(allocation));
    for(int i=0; i<n_processes; i++){
        int bestIdx = -1;
        for(int j=0; j<m_blocks; j++){
            if(blocks[j] >= processSize[i]){
                if(bestIdx == -1 || blocks[j] < blocks[bestIdx]){
                    bestIdx = j;
                }
            }
        }
        if(bestIdx != -1){
            allocation[i] = bestIdx;
            blocks[bestIdx] -= processSize[i];
        }
    }
    printf("\n--- Best Fit Allocation ---\n");
    printAllocation(processSize, n_processes, allocation);
}
void worstFit(int blockSize[], int m_blocks, int processSize[],
        int n_processes){
    int allocation[MAX_PROCESSES];
    int blocks[MAX_BLOCKS];
    memcpy(blocks, blockSize, sizeof(blocks[0]) * m_blocks);
    memset(allocation, -1, sizeof(allocation));
    for(int i=0; i<n_processes; i++){
        int worstIdx = -1;
        for(int j=0; j<m_blocks; j++){
            if(blocks[j] >= processSize[i]){
                if(worstIdx == -1 || blocks[j] > blocks[worstIdx]){
                    worstIdx = j;
                }
            }
        }
    }
}
```

```
    if (worstIdx != -1) {
        allocation[i] = worstIdx;
        blocks[worstIdx] -= processSize[i];
    }
}
printf("\n--- Worst Fit Allocation ---\n");
printAllocation(processSize, n_processes, allocation);
}

int main() {
    int m_blocks, n_processes;
    int blockSize[MAX_BLOCKS], processSize[MAX_PROCESSES];

    printf("Enter the number of memory blocks: ");
    scanf("%d", &m_blocks);
    printf("Enter the size of each block:\n");
    for (int i = 0; i < m_blocks; i++) {
        printf("Block %d: ", i + 1);
        scanf("%d", &blockSize[i]);
    }
    printf("\nEnter the number of process: ");
    scanf("%d", &n_processes);
    printf("Enter the size of each process:\n");
    for (int i = 0; i < n_processes; i++) {
        printf("Process %d: ", i + 1);
        scanf("%d", &processSize[i]);
    }
    firstFit(blockSize, m_blocks, processSize, n_processes);
    bestFit(blockSize, m_blocks, processSize, n_processes);
    worstFit(blockSize, m_blocks, processSize, n_processes);
    return 0;
}
```

O/P:

```
Enter the number of memory blocks: 5.
Enter the size of each block:
Block 1 : 100
      2 : 500
      3 : 800
      4 : 300
      5 : 600
Enter number of process : 4
Enter the size of each process.
process 1 : 212
        2 : 417
        3 : 112
        4 : 426
```

---- First Fit allocation ---

Process No	Size	block No.
1	212	2
2	417	5
3	112	2
4	426	Not allocated.

---- Best Fit allocation- --

Process no	Size	block no
1	212	4
2	417	2
3	112	3
4	426	5

---- Worst Fit allocation- --

Process no	size	Block no
1	212	5
2	417	2
3	112	5
4	426	Not allocated.

lab-10

Page Replacement algo

```
# include <stdio.h>
# define MAX 30

void FIFO(int pages[], int n, int frames){
    int frame[MAX], front=0, count=0;
    int pageFaults=0;
    for(int i=0; i<frames; i++)
        frame[i]=-1;
    for(int i=0; i<n; i++){
        int found=0;
        for(int j=0; j<frames; j++){
            if(frame[j]==pages[i]){
                found=1;
                break;
            }
        }
        if(!found){
            frame[front]=pages[i];
            front=(front+1)%frames;
            pageFaults++;
        }
        printf("\nStep %d:", i+1);
        for(int j=0; j<frames; j++)
            printf("%d", frame[j]);
    }
    printf("\nFIFO page Faults=%d\n", pageFaults);
}

int FindOptimal(int pages[], int frame[], int n, int idx, int frames
    int res=-1, farthest=idx;
    for(int i=0; i<frames; i++){
        int j;
        for(j=idx; j<n; j++){
            if(frame[i]==pages[j]){
```

```
                farthest = j;
                res = i;
            }
            break;
        }
    }
    if (j == n)
        return i;
    }
    return (res == -1) ? 0 : res;
}
void optimal(int pages[], int n, int frames){
    int frame[MAX];
    int pageFaults = 0;
    for (int i=0; i< Frames; i++)
        Frame[i] = -1;
    for (int i=0; i<n; i++){
        int found=0;
        for (int j=0; j< frames; j++){
            if (frame[j] == pages[i]){
                found = 1;
                break;
            }
        }
        if(!found){
            int replaceIdx = -1;
            for (int j=0; j< frames; j++){
                if (frame[j] == -1){
                    replaceIdx = j;
                    break;
                }
            }
            if (replaceIdx == -1)
                replaceIdx = FindOptimal(pages, frame, n, i+1, fra
            frame[replaceIdx] = pages[i];
            pageFaults++;
        }
        printf("\nStep %d:", i+1);
        for (int j=0; j< frames; j++)
            printf("%d", frame[j]);
    }
    Printf("\n\n optimal page faults = %d\n", pageFaults);
}
```

```
void LRU(int pages[], int n, int frames){
    int frame[MAX], time[MAX];
    int counter=0, pageFaults=0;

    for(int i=0; i<frames; i++){
        frame[i] = -1;
        time[i] = 0;
    }
    for(int i=0; i<n; i++){
        int found=0;
        for(int j=0; j<frames; j++){
            if(frame[j]==pages[i]){
                counter++;
                time[j] = counter;
                found = 1;
                break;
            }
        }
        if(!found){
            int replaceIdx = -1;
            for(int j=0; j<frames; j++){
                if(frame[j]==-1){
                    replaceIdx = j;
                    break;
                }
            }
            if(replaceIdx==-1)
                replaceIdx = findLRU(time, frames);
            frame[replaceIdx] = pages[i];
            counter++;
            time[replaceIdx] = counter;
            pageFaults++;
        }
        printf("\n Step %d:", i+1);
        for(int j=0; j<frames; j++)
            printf("%d ", frame[j]);
    }
    printf("\n\n LRU page Faults = %d\n", pageFaults);
}
```

```
int main() {
  int pages[MAX], n, frames, choice;
  printf("Enter number of pages: ");
  scanf("%d", &n);
  printf("Enter page reference string:\n");
  for (int i=0; i<n; i++)
    scanf("%d", &pages[i]);
  printf("Enter no of frames: ");
  scanf("%d", &frames);
  while(1) {
    printf("\n\n--- Page Replacement Menu ---\n");
    printf("1. FIFO\n");
    printf("2. Optimal\n");
    printf("3. LRU\n");
    printf("4. Exit\n");
    printf("Enter choice: ");
    scanf("%d", &choice);
    switch (choice) {
      case 1:
        FIFO(pages, n, frames);
        break;
      case 2:
        Optimal(pages, n, frames);
        break;
      case 3:
        LRU(pages, n, frames);
        break;
      case 4:
        return 0;
      default:
        printf("Invalid choice!\n");
    }
  }
}
```

O/P Enter no of frames: 3.
Enter length of replacement string: 7
Enter reference string space-separated:
1 3 0 3 5 6 3

--- FIFO Page replacement ---
Ref 1: [1 -1 -1]
Ref 3: [1 3 -1]
Ref 0: [1 3 0]
Ref 3: Hit → [1 3 0]
Ref 5: [5 3 0]
Ref 6: [5 6 0]
Ref 3: [5 6 3]
Total page Faults: 6
Total page hits: 1

--- optimal page replacement ---
Ref 1: [1 -1 -1]
Ref 3: [1 3 -1]
Ref 0: [1 3 0]
Ref 3: Hit → [1 3 0]
Ref 5: [5 3 0]
Ref 6: [5 3 6]
Ref 3: Hit → [5 3 6]
total Page Faults: 5
total page Hits: 2.

--- LRU Page Replacement ---
Ref 1: [1 -1 -1]
Ref 3: [1 3 -1]
Ref 0: [1 3 0]
Ref 3: Hit → [1 3 0]
Ref 5: [5 3 0]
Ref 6: [5 3 6]
Ref 3: Hit → [5 3 6]
total Page Faults: 5
total Page Hits: 2.